

Java I/O Streams

Java provides a rich set of I/O (Input/Output) classes to read and write data. These classes are part of the `java.io` package and are based on the concept of streams. A stream represents a sequence of data and can be used for reading from or writing to a data source or destination.

Types of Streams

In Java, streams are divided into two main categories:

1. **Byte Streams:** Work with binary data (8-bit bytes)
2. **Character Streams:** Work with character data (16-bit Unicode characters)

Let's explore various types of streams in detail:

1. File Streams

Handle reading/writing data directly to/from files.

- **Byte Streams:**
 - **FileInputStream:** Reads bytes from a file.
 - **FileOutputStream:** Writes bytes to a file.
 - **Use Case:** Binary files (e.g., images, PDFs).

Example:

```
import java.io.*;

public class FileStreamExample {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("input.txt");
             FileOutputStream fos = new
FileOutputStream("output.txt")) {
```

```

        int byteData;
        while ((byteData = fis.read()) != -1) {
            fos.write(byteData);
        }
        System.out.println("File copied successfully.");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Character Streams:

- **FileReader**: Reads characters from a text file.
- **FileWriter**: Writes characters to a text file.
- **Use Case**: Text files with proper character encoding.

Example:

```

import java.io.*;

public class FileReaderWriterExample {
    public static void main(String[] args) {
        try (FileReader fr = new FileReader("input.txt");
            FileWriter fw = new FileWriter("output.txt")) {

            int charData;
            while ((charData = fr.read()) != -1) {
                fw.write(charData);
            }
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

2. Object Streams

Serialize/deserialize Java objects (convert objects to bytes and vice versa).

- **ObjectInputStream**: Reads serialized objects.
- **ObjectOutputStream**: Writes serialized objects.
- **Requirements**: Objects must implement a Serializable interface.

ObjectInputStream and ObjectOutputStream

These streams are used for serialization and deserialization of objects.

Example:

```
import java.io.*;

class Person implements Serializable {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}

public class ObjectStreamExample {
    public static void main(String[] args) {
        Person person = new Person("Alice", 30);

        // Serialization
```

```

        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("person.ser"))) {
            oos.writeObject(person);
            System.out.println("Object serialized successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Deserialization
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("person.ser"))) {
            Person deserializedPerson = (Person) ois.readObject();
            System.out.println("Deserialized object: " +
deserializedPerson);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

3. ByteArray Streams

Use byte arrays as the source/destination in memory.

- **ByteArrayInputStream**: Reads from a byte array.
- **ByteArrayOutputStream**: Writes to a byte array.
- **Use Case**: In-memory data manipulation (e.g., network protocols).

ByteArrayInputStream and ByteArrayOutputStream

These streams treat an array of bytes as a stream of bytes.

Example:

```
import java.io.*;

public class ByteArrayInputStreamExample {
    public static void main(String[] args) {
        String data = "Hello, ByteArray Streams!";
        byte[] bytes = data.getBytes();

        try (ByteArrayInputStream bais = new
ByteArrayInputStream(bytes);
            ByteArrayOutputStream baos = new
ByteArrayOutputStream()) {

            int byteData;
            while ((byteData = bais.read()) != -1) {
                baos.write(Character.toUpperCase((char) byteData));
            }

            System.out.println("Original: " + data);
            System.out.println("Transformed: " + baos.toString());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

4. Buffered Streams

Buffered streams improve I/O performance by reducing the number of I/O operations through buffering.

BufferedInputStream and BufferedOutputStream

These are byte streams that add buffering to the underlying input and output streams.

Example:

```
import java.io.*;

public class BufferedStreamExample {
    public static void main(String[] args) {
        try (BufferedInputStream bis = new BufferedInputStream(new
FileInputStream("input.txt"));
            BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream("output.txt"))) {

            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = bis.read(buffer)) != -1) {
                bos.write(buffer, 0, bytesRead);
            }

            System.out.println("File copied successfully with
buffering.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

BufferedReader and BufferedWriter

These are character streams that add buffering to the underlying reader and writer.

Example:

```
import java.io.*;

public class BufferedReaderWriterExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new
FileReader("input.txt"));
            BufferedWriter bw = new BufferedWriter(new
FileWriter("output.txt"))) {

            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }

            System.out.println("File copied successfully with
buffering.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

5. Reader and Writer

Reader and Writer are abstract classes for reading and writing character streams.

InputStreamReader and OutputStreamWriter

These classes bridge between byte streams and character streams.

Example:

```
import java.io.*;

public class ReaderWriterExample {
    public static void main(String[] args) {
        try (Reader reader = new InputStreamReader(new
FileInputStream("input.txt"), "UTF-8");
            Writer writer = new OutputStreamWriter(new
FileOutputStream("output.txt"), "UTF-8")) {

            char[] buffer = new char[1024];
            int charsRead;
            while ((charsRead = reader.read(buffer)) != -1) {
                writer.write(buffer, 0, charsRead);
            }

            System.out.println("File copied successfully with
character encoding.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```


Key Considerations

1. **Exception Handling:** Always handle `IOException` and use **try-with-resources** for auto-closing streams.
2. **Performance:** Use buffered streams for large files to minimize I/O operations.
3. **Character Encoding:** Explicitly specify encoding (e.g., `StandardCharsets.UTF_8`) to avoid platform-dependent issues.
4. **Decorator Pattern:** Java I/O streams can be chained (e.g., `BufferedReader` wraps `FileReader`).

Best Practices

- Use **try-with-resources** (Java 7+) to ensure streams are closed.
- Prefer `BufferedReader/BufferedWriter` for text files.
- Use `flush()` to force data to be written (e.g., before closing a `BufferedOutputStream`).
- For object serialization, declare `serialVersionUID` to manage version control.

Summary of Java I/O Streams

Stream Type	Class Name	Use Case
Byte Streams	<code>FileInputStream</code>	Reading binary data from a file
	<code>FileOutputStream</code>	Writing binary data to a file
	<code>ObjectInputStream</code>	Reading a serialized object

	<code>ObjectOutputStream</code>	Writing a serialized object
	<code>ByteArrayInputStream</code>	Reading data from a byte array
	<code>ByteArrayOutputStream</code>	Writing data to a byte array
Character Streams	<code>FileReader</code>	Reading text data from a file
	<code>FileWriter</code>	Writing text data to a file
	<code>BufferedReader</code>	Reading text efficiently
	<code>BufferedWriter</code>	Writing text efficiently